INTERNATIONAL
UNIVERSITY OF
APPLIED SCIENCES

IU International University of Applied Sciences

Berlin

A PROJECT REPORT

on

Smart Farming Platform

Course: Project Software Engineering (DLMCSPSE01)

Submitted by

Navjot Kaur

4252310

In partial fulfillment of the requirements for the I Semester

M.Sc in Computer Science

Under the Guidance of

Prof. Dr. Holger Klus

Date of Submission: 12-04-2026

Table of Contents

Table of Figures ..... *iii*

List of Tables ..... *iv*

Abstract ..... *v*

Acknowledgement ..... *vi*

1. *Introduction* *1*

1.1 Objectives and Scope 1

1.2 Target Group 1

1.3 Project Risks and Mitigation Strategies 2

1.4 Functional Requirements 2

1.5 Non-functional Requirements 3

2. Literature Survey *4*

2.1 Smart Farming using IoT and Machine Learning 4

2.2 AI-Based Crop Prediction Systems 4

2.3 Use of UAVs in agriculture 4

2.4 Agricultural Data Systems and Integration 5

2.5 Remote Sensing in agriculture 5

3 Technological Stack *6*

4 Software Development Methodology *7*

5 System Design and Architecture *8*

5.1 System Context and Scope 8

5.2 System Architecture Diagram 8

5.3 UML Diagram 9

5.4 Data Flow Diagram 10

6 Implementation *12*

6.1 Frontend Implementation 12

6.2 Backend Implementation 12

6.3 Database Implementation 12

6.4 AI and Machine Learning Implementation 13

6.5 Integration with External APIs 13
6.6 Alerts and Notification System 13
6.7 Deployment and System Execution 13
7 System Testing ***14***
7.1 Unit Testing 14
7.2 Integration Testing 14
7.3 Test Cases Summary 15
8 Challenges ***16***
8.1 Technical Challenges 16
8.2 Data Challenges 16
8.3 System and Deployment Challenges 16
9 Conclusion and Future work ***17***
9.1 Conclusion 17
9.2 Future Work 17
10. Results ***18***
11. Glossary ***21***
GitHub and Deployment Link ***21***
References ***22***

Table of Figures

Figure 1: System Architecture of Smart Farming Platform 9
Figure 2: UML Component Diagram of Smart Farming System 10
Figure 3: Data Flow diagram of Smart Farming System 11
Figure 4: Dashboard Overview 18
Figure 5: Crop Management Page 18
Figure 6: Crop health Analysis 19
Figure 7: Yield Prediction Module 19
Figure 8: Weather and Soil Data 20

List of Tables

Table 1: Project Risks and Mitigation Strategies 2
Table 2: Technology stack of Smart Farming Platform 6
Table 3: Test Cases 15

Abstract

Agriculture remains a foundational element of global economies which represents the core profession for millions around the globe. The challenges facing farmers particularly small scale, rural and traditional ones remain formidable and persistent which ranges from unpredictable weather and decreasing soil productivity to pervasive pest issues and drastically unstable market prices. Compounded by the chronic underutilization of real-time, data driven tools, decision-making remains a tremendously complex and arduous task.

The Smart Farming Platform is a web application solution that bridges this gap by integrating key agricultural functionalities into a cohesive and unified digital ecosystem. The application leverages AI-based crop health assessment, machine-learning-informed yield forecasting, real-time monitoring of weather and soil conditions, real-time market price visualization and analysis and a proactive smart alert and notification system to enable intelligent decisions and ultimately, increased profitability for farmers.

The application employs a React-based frontend coupled with Tailwind CSS and UI and a Python FastAPI-based backend with an PostgreSQL database to facilitate a responsive user experience and efficient data processing. The architecture of system is based on Agile methodologies and structured in three phases such as Conception, Development and Finalization. Each phase resulted in iterative, working software increments that were validated against established requirements.

Future work will include a scalable design allowing for eventual cloud deployment, expanded AI capabilities and potential integration with external agricultural APIs. This report outlines the complete software development life cycle which includes requirement analysis, system design, implementation, testing and a concluding retrospective of lessons learned.

Acknowledgement

It gives me immense pleasure to present the Project Report on the **Smart Farming Platform** undertaken under the syllabus of IU International University of Applied Sciences, Berlin in partial fulfillment of the requirement for the 1st semester for the Master's program in computer sciences.

I am extremely thankful to my guide Mr. Holger Klus, Professor IU International University of Applied Sciences, Berlin, for the timely guidance and timely suggestions at every step of this project, not forgetting his consistent guidance all throughout the project work.

I am also thankful to my friends and family for all their motivation and support.

Navjot Kaur

Student ID: 4252310

1. Introduction

Worldwide various industries, including agricultural practices are in the process of undergoing profound digital transformation. Even though the agriculture plays a pivotal role in the economy and ensuring food security worldwide, access to smart technology is denied to numerous farmers especially those from rural and developing countries which hinders their success and profit levels.

The **Smart Farming Platform** is one of such platforms in the form of intelligent Web system to consolidate, store, retrieve and visualize information about crop management, crop health and detection of issues, AI based yield prediction, weather and soil information, market price trends and notifications in a central and user-friendly dashboard that empowers farmers with evidence based decisions making based on the best combination of the real data and the AI capabilities.

1.1 Objectives and Scope

The primary objective of the Smart Farming Platform is to serve as the digital gateway that gives farmers agriculture intelligence and decision support systems with the help of artificial intelligence technology. The precise objectives include the following:

- Offer a single dashboard for farmers to manage and visualize farm-specific information and operations and to enable an AI-based crop health diagnosis by allowing farmers to report crop details and symptoms with AI generated recommendations.
- Support an AI-based yield forecasting that leverage crop features such as variety, planting date, area and location.
- Provide farmers with a dashboard displaying real-time farm intelligence such as weather, soil data and market prices also provide secure access for users through an authentication system and role-based security to the Smart Farming Platform features and services.

1.2 Target Group

Farmers who are either small or medium sized and are looking for affordable and practical solutions for managing farms and farms as well as having limited access to advanced digital solutions are one of the major user groups that can use this platform ultimately through simple interfaces with minimal technical ability. Within this user group, cooperative and other types of community based agricultural organizations can also share and access (crop and market) and climate related information in a centralized mode along with agricultural advisors, international NGOs and government agencies who help support smallholder farms by analyzing crop production and offering guidance about best farming practices. Developers and researchers can also be a secondary user group and can potentially develop additional integrations into the platform via the use of other data and/or AI-based models.

1.3 Project Risks and Mitigation Strategies

Table 1: Project Risks and Mitigation Strategies

Risk	Description	Mitigation strategy
Technical Risk	Latency and undefined response format possible using AI APIs for example weather or soil data	Utilize structured prompt templates and default responses; incorporate retry mechanism in backend.
Data Accuracy Risk	Poor weather and soil data can affect accuracy of the recommendation	Tag simulated information clearly on user interface; plan API integration during finalization process.
Adoption Risk	Target farmer not willing to adapt the system due to lack of computer literacy	Create a simple easy to use interface driven by icons and less actions required provide onboarding assistance in the form of hints on dashboard.
Scalability Risk	No simultaneous writing supported by PostgreSQL when multiple users are using performance falls significantly	Consider documentation as technical debt and switch to PostgreSQL prior to launching into production
Schedule Risk	In event of some unpredicted failure can be challenging in terms of repair or recovery in case of only one developer working on the product.	Agile methodology should be employed, with proper scope defined for each sprint, concentrating on important features while putting off others.

1.4 Functional Requirements

- **User Authentication:** Users must log into their account to enter any type of data on this platform.
- **Data Management:** Farmers add their data on the type of crop when the crop was to be planted and how much water will be needed for the crop.
- **Weather Information System:** Current weather and weather forecasts are available throughout the entire duration of the crop cycle.
- **Crop Recommendations Feature:** Crop recommendations and how to plant them will be shown depending on climate and soil condition.

1.5 Non-functional Requirements

- **Usability:** Interface of the system should be very simple and easy to use even by non-technical individuals.
- **Performance:** The platform will take no longer than a few seconds to respond after the interaction.
- **Security:** The entire sensitive information will be encrypted as role and rights management will protect sensitive data and information.
- **Scalability:** The platform will scale to accept the increase of users and data with reasonable efforts
- **Reliability:** The platform will be available with no more than acceptable levels of time downtime.

2. Literature Survey

2.1 Smart Farming using IoT and Machine Learning

A smart farming system that uses machine learning algorithms along with the Internet of Things devices to optimize and guide the decisions for agricultural production. The proposed system involves data collection in real-time such as measuring factors such as temperature, humidity and soil moisture using sensors placed in the fields which then undergoes further processing using various ML driven algorithms to generate recommendations for the farmers (Patel and Sharma, 2021).

The paper suggests that such continuous sensing of environmental parameters could help farmers enhance crop productivity and conserve natural resources. It implies that data collection along with the usage of automation is quite important in modern farming. The challenge is the necessity of the physical IoT devices in their system to operate it (Patel and Sharma, 2021). This issue reinforces the advantages of the proposed Smart Farming Platform as it works on less complex physical setups without being entirely dependent on them for farmers who run small-scale agricultural activities.

2.2 AI-Based Crop Prediction Systems

This method of artificial intelligence can be helpful in identifying predicting how the crop production might be like and may provide for the farmer to understand that part. By using models that use data from weather and different features of crops for identifying and analyzing the yield based on the present weather which is an effective process to determine the crop which has to grow at specific climate that help in planning the crop cultivation which is also supported through the usage of various software which takes data and predicts the crop yield and how and when and how much the yield can produce(Kumar & Mishra, 2023).

2.3 Use of UAVs in agriculture

Additional aspects of modern agricultural systems that can provide similar information include unmanned aerial vehicles. UAVs can fly over land, collecting images with high levels of resolution, that can be studied for areas of growth stress, disease or other anomalies of interest (Zhang & Kovacs, 2012).

Image acquisition for such an endeavor can be incredibly precise and data intense, and thus it can be very useful to large-scale agriculture projects that will be able to immediately assess problem areas of a farm. The disadvantages to acquiring such data can be the cost, as well as the need for expertise and hardware that may be beyond the means of a small farmer, supporting the usefulness of remote sensing via satellites and online data interfaces. (Zhang & Kovacs, 2012)

2.4 Agricultural Data Systems and Integration

Effective decisions can be ensured if more information regarding agriculture sources can be integrated together. With different information of weather stations, data bases related to the soil and files recording crop development, a wide vision to obtain detailed farming information can be got (Jones et al., 2017).

Furthermore, these data integration applications could provide supports to more intelligent planning, saving resources effectively, increasing outputs. But dealing with big data with multiple source systems also raises challenges when systems are designed effectively (Jones et al., 2017). The need for systems designed to deal with this data in integrated form for better presentation as illustrated with Smart Farming Platform.

2.5 Remote Sensing in agriculture

Modern farming relies on technology such as remote sensing technologies that acquire large-scale data from the view of satellites and they acquire this data to determine the health of crops, health of soils, the rate of natural change with time (Mulla, 2013). These approaches will determine the changing agricultural patterns throughout long term agriculture and use resources in a better way. However, the interpretation satellite data is very challenging that need technical knowledge (Mulla, 2013). Combining it into the convenient form for all agricultural workers is also one part of Smart Farming Platform project.

3 Technological Stack

The choice of technologies is an endeavor in trying to balance developer productivity, ecosystem size and domain maturity. Below is the rationale for each of them:

Table 2: Technology stack of Smart Farming Platform

Component	Technology	Rationale
Frontend UI	React	Reusability through components. crop cards, alert badge. Ecosystem size and a strong precedent for agricultural app usage made React the choice over Vue or Angular. The flexibility and ease of adoption compared to a framework is attractive to a new project.
UI Styling	Tailwind CSS + Shadcn UI	Speed by using utility-first classes for rapid iteration and development without the overhead of custom CSS. Shadcn UI brings easily composable and accessible components to speed things up.
Data Visualization	Recharts	A lightweight react charting library suitable for plotting trend charts and area charts in the dashboard. Lighter weight and more performant for react components than Chart.js, while more seamlessly integrating into react state than Chart.js.
Backend Framework	FastAPI (Python)	Asynchronous first framework which automatically generates API documentation from openAPI specs. It is an ideal framework for potential integration with future ML/AI models as python is a mature language for data science libraries.
Database	PostgreSQL	Production ready RDBMS with great concurrency writes and complex queries capabilities. Will replace PostgreSQL in the finalization phase.
Database (Prototype)	PostgreSQL	Zero config, file-based database system is ideal for getting a prototype up quickly for academic submission. The main reason for choosing this was because of ease of use though with the understanding this would need to change for production.

4 Software Development Methodology

This will be developed using Agile, following the Scrum approach, modified to suit a solo development environment. This was chosen above a plan-driven method as:

• **Flexibility:** Agricultural software requirements were likely to change and evolve as the developer learnt more about the requirements of farmers and as the professor provided feedback on the project.

• **Iterative Delivery:** By taking a short sprint-based approach of just two weeks per sprint, working and testable pieces of software would be available throughout the project as opposed to only at the end.

• **Risk Reduction:** By testing the integration between components as early as possible potential problems with integration would be raised and resolved before becoming too serious.

Course alignment the three phases of the portfolio - Conception, Development and Finalization - fit naturally into milestones within an agile process with Sprint Reviews and retrospectives being held at the conclusion of each. The three phases act as boundaries of an iteration. Within each iteration, a backlog of user stories will be built to be implemented for that iteration. The stories will be developed, then tested, followed by a reflection. A kanban based board was used to track the state of stories in a simple spreadsheet.

Given that the project is a solo one, the developer will be acting as the Product Owner who is responsible for prioritization, the Scrum Master and the Developer who is responsible for delivering the stories. All design decisions will be documented in a project repository in the form of Architecture Decision Records (ADRs).

5 System Design and Architecture

5.1 System Context and Scope

The platform acts as the focal point of a sociotechnical system integrating farmers with outside data and AI facilities. Below are the actors and outside programs we can include with our approach.

- **Farmer (Primary User):** Has interface via the web internet browser; handles plants, ask of AI evaluation, views climate and promote values and handles alarms.
- **AI/LLM Service:** An OpenAI compatible REST interface utilized to ingest the input structured concerns from the web client as well as return findings of crop wellness analysis and yield forecasting.
- **Weather API:** Uses real-time as well as projected climate data relevant to the user's farm location (implementation planned to use an OpenWeatherMap API.
- **Market Data Source:** Gives crop rate indices to fill the Market Rates module.

Our method system limit includes the Net frontend application written making use of React along with the FastAPI backend application. All data will certainly be persistent to the PostgreSQL backend or PostgreSQL backend. Outside sources are accessed via HTTPs via the web servers from the web application.

5.2 System Architecture Diagram

The architecture is broken down into five core components: User interface, backend server, ai service, external APIs, database. The main aim is for the system to gather relevant data, process it and make useful information for the farmer and to deliver the result.

- **Presentation Layer (Frontend – React.js)**
  The presentation layer is basically the user facing UI developed using React. It aims to present an intuitive, easy-to use interface through which the farmer can access features to. For example, add new crops or delete existing ones then check crop health status and get yield projections. Access weather, soil and market related data Send and receive requests to or from the backend It displays the outcome of these processes to the user in an easily understandable and visually pleasing manner.

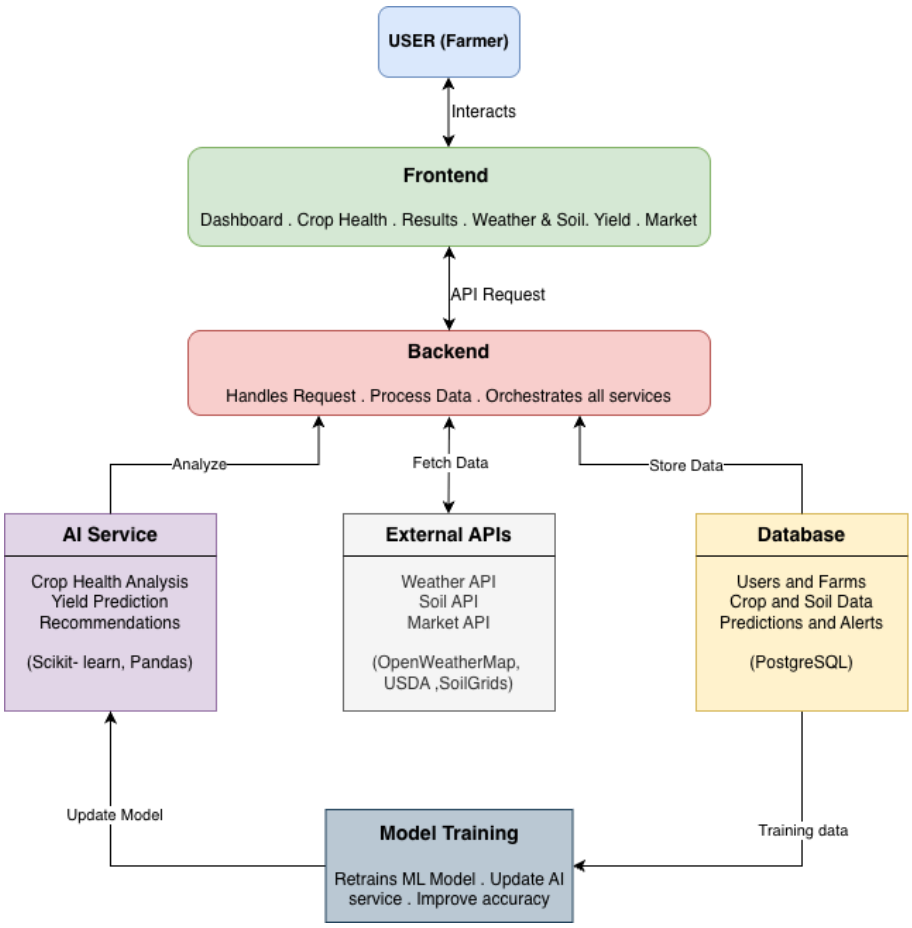

Figure 1: System Architecture of Smart Farming Platform

- **Application Layer (Backend - FastAPI)**

  The application layer, implemented using FastAPI, forms the backend of the system. It receives all requests from the user and is responsible for performing operations such as validating user requests, managing data, communicate with AI services and API services which ensures communication between the frontend, database and the outside services and the creation of the predicted outputs and alarms or recommendations.

- **Data Layer (PostgreSQL)**

  Data Layer is responsible for storage and management of all data related to the system such as users, crops, predictions and alerts. Relational Database like PostgreSQL is used for storing data securely and efficiently. All the data for analysis or decision making are accessed from this layer.

5.3 UML Diagram

The Smart Farming Platform main components are shown in the UML Diagram above. System is composed by a Frontend, Backend modules and Database and other external APIs interacting each other to offer some services. User communicates with the system through Frontend which sends

requests to Backend like add crop, check crop health, show predictions etc. Backend module is composed by different modules such as Authentication module, Crop module, AI service, Weather and soil module, Market module and Notification/Alert module. each one handles a specific aspect of the service. Back-end receives user request, then connects with data base to get or store information about users, crops or predictions and connect to external APIs such as weather API, soil analysis API, market API and AI service API to fetch data, perform analysis and so on. Finally, Backend will respond to Frontend and will present user the result in terms of insights, predictions and alerts.

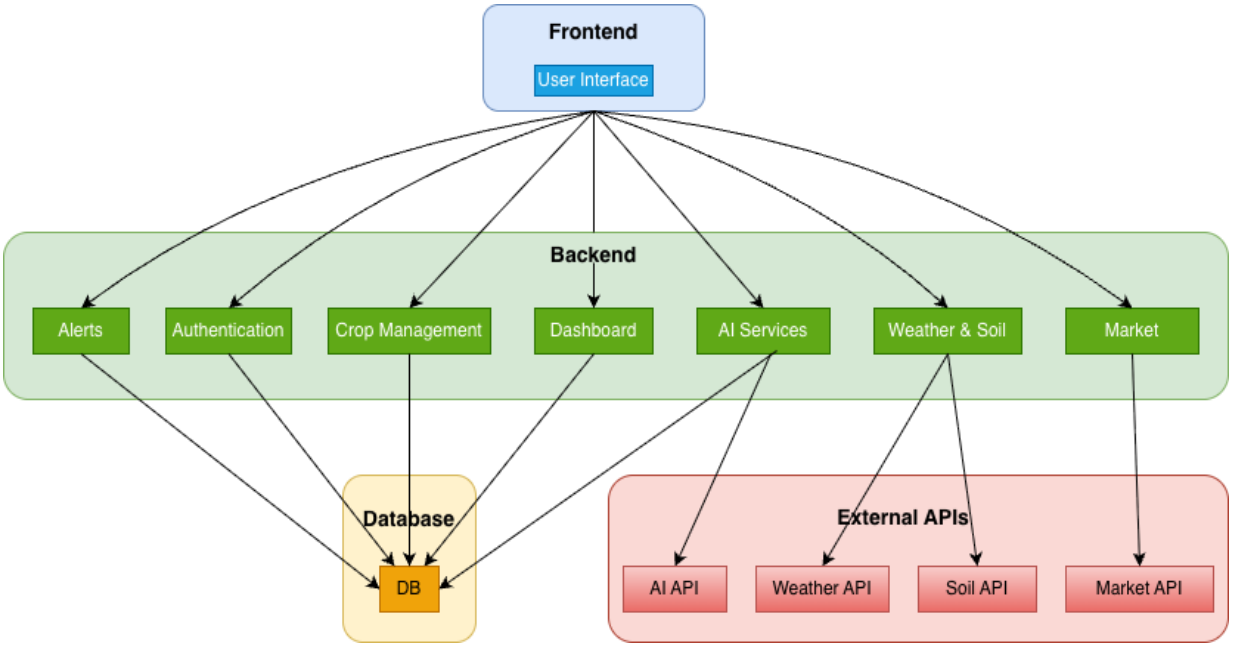

Figure 2: UML Component Diagram of Smart Farming System

5.4 Data Flow Diagram

The data flow diagram depicts how the data will flow into the Smart Farming Platform, and how the different processes would engage with each other. Login will begin with the farmer sending login credentials which will be validated by the system login process against the system database which after validation will generate an authentication token for the user and the system will grant access to its functions. On successful login, the user will be able to Input their crops details and pass them through the crop management process, which stores and retrieve data from the data store to and from the crop details. These details are also transmitted to the AI analysis module.

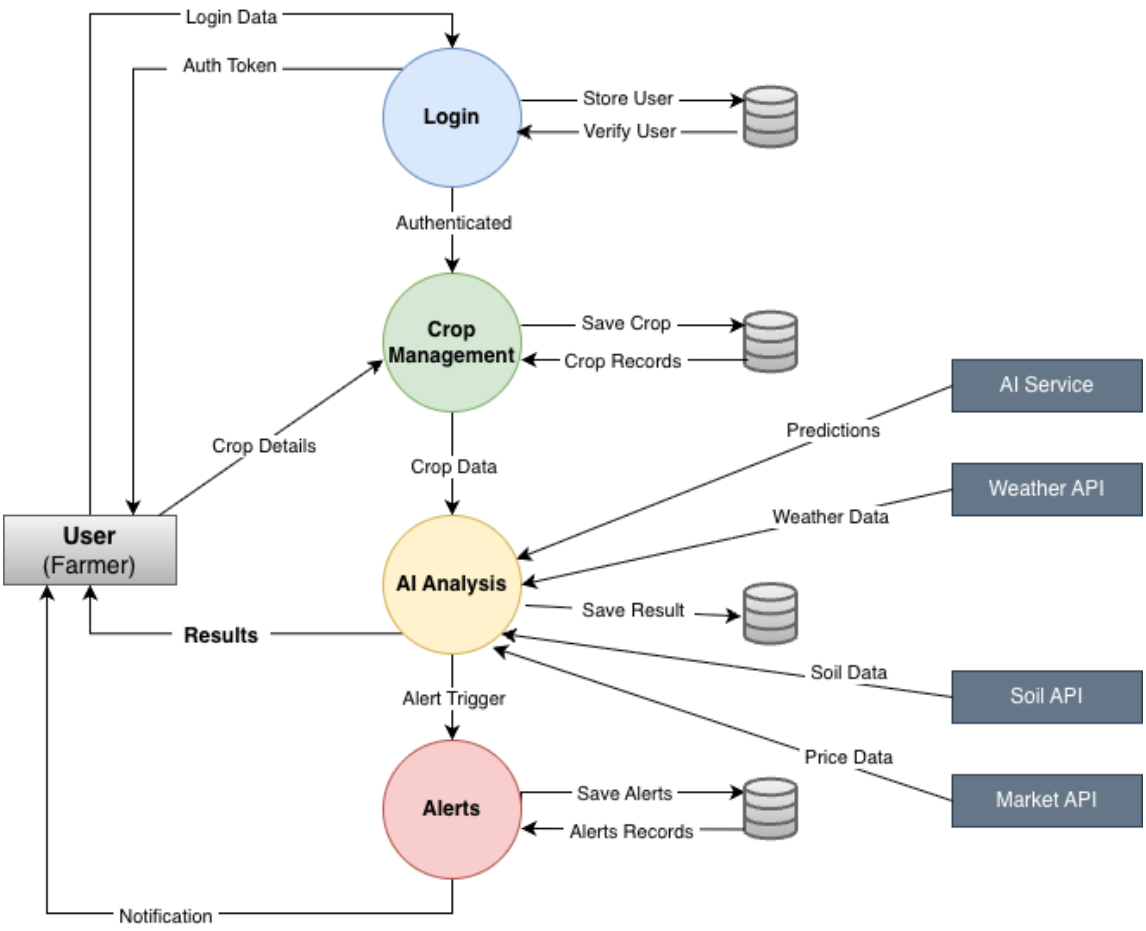

Figure 3: Data Flow diagram of Smart Farming System

The AI analysis module is concerned with the analysis of the crop health, crop yield predictions using other parameters like data from weather APIs, market prices from external APIs, data from soil sensors. The analyzed details will then be recorded into the data store and a result sent back to the user. The alerting module sends out notifications and records into the database based on whether a critical condition is observed in any of the crops or environment. Such alarms also receive communication from the other modules of the system and they are transmitted to the user to alert him about any pertinent and timely issue like risk associated with any of his crops.

6 Implementation

During the implementation the proposed system is implemented in the real form to build a smart farming system – the Smart Farming platform. It is a full-stack web application: the designed system is built as a complete full stack web application incorporating frontend, backend, database, AI models and other external APIs. This helps in the implementation of the system with modules including crop management, AI-based system analysis, weather and soil data ingestion, alerts etc. This system is designed with simple user interface and easy operation for farmers.

6.1 Frontend Implementation

The frontend of the system is implemented using the popular and efficient react library and offers an interactive responsive user interface that is friendly for even users with no technical knowhow and basic. The frontend comprises of pages like Dashboard, My Crops, Crop Health, Yield Prediction, Weather and Soil, Market prices and alerts. Each page is intended to do specific actions like on the crop management page, users can add crops details, on the Crop Health page, user input the crop health symptoms and view the results of analysis based on those symptoms. The frontend can send information and receive information through the backend by making API calls through Axios and provides various types of interactive graphs and charts to visualize data.

6.2 Backend Implementation

This backend application built using FastAPI, a high-performance python framework is the brains of the operation that contains all business logic for the application. The backend consists of multiple modules such as authentication, crop Management, AI services, weather and soil Services, market services, alerts and dashboard services. This modularization ensures that each component is responsible for a specific part of the functionality and exposes individual API endpoints for the frontend to communicate with. In the Authentication module security of the backend is maintained with the help of JWT tokens and password hashing with bcrypt which in turn is used to maintain the connectivity between backend and external APIs.

6.3 Database Implementation

The system stores and manages all data within the system using a relational database. For database persistence PostgreSQL will be used as the database system as well but PostgreSQL may be employed at the prototype phase for its simplicity. The data can contain tables like users, crops, alerts and predictions which will contain relevant data like users, crops and any kind of predictions or other system results also notifications for the user. CRUD (create, Read, update, delete) operations will be performed by the backend against the database.

6.4 AI and Machine Learning Implementation

AI in the system uses python frameworks including Pandas and Scikit-learn for analysis of crops and output intelligent results. Based on the symptoms and environmental factors entered by users, this system calculates the crop health. Also, by utilizing weather patterns and crop and soil data, crop yield is computed using our system. It provides solutions that can contribute to increasing yield. NDVI (normalized differential vegetation index) derived from satellite images are also utilized to further evaluate the vegetation to predict it even more accurately.

6.5 Integration with External APIs

The Smart Farming Platform combines several external APIs to get necessary data to conduct analysis at real-time. Weather data from sources like OpenWeatherMap can be fetched with real-time temperature, humidity and weather forecast. Soil data from sources like SoilGrids can also be acquired with details of the soil quality and characteristics. Market data is retrieved from sources to understand crop prices and trends. NASA earth data is integrated with features like remote sensing with NDVI for analyzing imagery from Earth surface. The data from these sources along with the internal data is analyzed by the backend to derive valuable insights.

6.6 Alerts and Notification System

The alerts module is designed to notify users regarding significant happenings such as weather conditions changing, risks associated with crops and suggestions that should be made. The alerts are generated using artificial intelligence analysis and from data provided by outside sources. All the alerts are stored in the database and presented on the frontend of the system for users to see. Users have access to the alerts so that they can remain up to date regarding their crops and agriculture conditions.

6.7 Deployment and System Execution

The software is implemented using a web application interface which allows it to be used via a web browser. Both the frontend and backend components reside in cloud computing systems that enables scalability. The software can support several concurrent users and deliver quick responses. This guarantees proper functioning of all its capabilities.

7 System Testing

The tests have been conducted at various levels to make sure that the Smart Farming Platform works effectively. Various forms of tests were conducted to confirm the performance of individual components and system integration. Testing has been useful in identifying and correcting any errors as well as improving the effectiveness and reliability of the system.

7.1 Unit Testing

Unit testing was done on the functions and different parts of the backend to ensure their accuracy. Unit testing is the process used to determine the correctness of a single function when isolated from other processes in the software. The utility functions in the backend, including the password encryption function, JWT generation and verification, and AI prompt creation were done using the pytest package. Each of the functions was tested using a positive case and a negative test case.

For instance, functions dealing with password encryption were checked to ascertain that the passwords are encrypted correctly. Functions for JWT generation and verification were also checked to ensure the validity of the tokens, especially their expiration time. The same applies to prompt creation for the AI service. Unit tests were run before merging any changes into the master branch.

7.2 Integration Testing

To confirm that the various modules of the system do indeed interact properly, integration testing was performed. This testing examined the interaction of different components of the system with one other. The focus of integration testing included: frontend requests; backend API calls; execution of database commands; use of third-party services. Integration testing performed on the API endpoints used the FastAPI Test Client class, which internally utilizes the HTTPX library to mimic making an HTTP request without spinning up a test server. This allowed for a great deal of efficiency while also providing a means to conduct controlled testing of backend services.

A few examples of integration testing that were performed include:

- Testing user registration and user login, validating authentication in both cases
- Adding, modifying, retrieving, and deleting crop records
- Testing interaction between backend services and the database when inserting and retrieving information from there
- Testing the AI service endpoints using mock responses for simulating predictive results

The result of the integration testing was proof of the proper passing of information between modules and that the expected behavior of the system occurs when two or more modules are part of the interaction.

7.3 Test Cases Summary

Table 3: Test Cases

TC ID	Test Description	Expected Result	Type	Result
TC-01	Register with valid email and password	HTTP 201, user created in DB	Integration	Pass
TC-02	Login with correct credentials	HTTP 200, JWT returned	Integration	Pass
TC-03	Access protected endpoint without token	HTTP 401 Unauthorized	Integration	Pass
TC-04	Create a crop record with all required fields	HTTP 201, crop stored in DB	Integration	Pass
TC-05	Delete a crop that belongs to another user	HTTP 403 Forbidden	Integration	Pass
TC-06	Request crop health analysis with valid crop and symptoms	AI analysis returned within 10s	E2E	Pass
TC-07	Dashboard displays correct crop count after adding crops	Crop count matches DB records	E2E	Pass
TC-08	Mark an alert as read	Alert status updated, unread count decreases	E2E	Pass
TC-09	Bcrypt password hash verification	Correct password verifies, wrong password reject	Unit	Pass
TC-10	JWT generation and expiry validation	Expired token rejected with 401	Unit	Pass

8 Challenges

8.1 Technical Challenges

Managing external API integration. There are several APIs required for fetching external resources including weather, soil and market data. Data is extracted in different formats which necessitates data transformation. Maintaining consistent and valid data across integrated APIs became a problem. AI feature integration such as Implementing several AI features like yield forecasting, soil analysis and soil and crop health was part of it. The selection of proper model and managing various kinds of user input during testing took up much time and research work. Frontend and Backend integration such as there are API call, response mechanisms and data synchronizing issues between UI and backend. Handling various kinds of responses, managing latencies on API and dealing with asynchronous operations created new levels of difficulties in integrating both components.

8.2 Data Challenges

System dependence on external data the system depends to a large extent on external data sources. In some situations, the data may be not complete or up to date or simply missing for areas. These impact on reliability of predictions and recommendations. The absence of sufficiently localized data Some variations may be due to differences of climate and soil condition between areas. Such situation limits accuracy when data are missing at specific locations.

8.3 System and Deployment Challenges

The structural aspect of the system which makes sure that all components gel well, has been another tough job. Coordinating and managing multiple modules like authentication, crop management, AI service and alerts. Deployment of the system was again challenging; making sure that both frontend and backend components work well on the cloud service provider.

9 Conclusion and Future work

9.1 Conclusion

The Smart Farming Platform successfully showcase the integration of emerging technologies such as web development, Machine Learning and data integration to assist farmers in agricultural decision making. The application built as a web application in an integrated smart platform incorporated various functions into a single system. The crop data can be accessed and managed on the platform with additional access to crop health, weather and soil information and yield predictions can be done. Use of ML algorithms with prediction made via external API helped to provide farmers with valuable insights. An easy to access GUI was created for users even with basic technical background. With its modular design, each section, frontend, backend, database and AI services have been appropriately working to enable the smart farm.

9.2 Future Work

Smart farming is an ideal option, but there is much room for improvement for a future version. It would be beneficial if IoT sensors would be incorporated in a real-time manner collecting data from the field regarding moisture, humidity and temperature etcetera.

Also, better crop analysis and higher certainty in harvest forecasting can be reached by using complex ML models and bigger data sets. Mobile access is possible by integrating a mobile app for smartphone access. Language support is important to accommodate global users. Satellite imagery (NDVI) is available to view the health of the field but can be used in a far more extensive way.

Scalability will be realized in the further phase by deploying the system to the cloud. Users are going to access it offline through their smartphone and receive tailored recommendations by the system.

10. Results

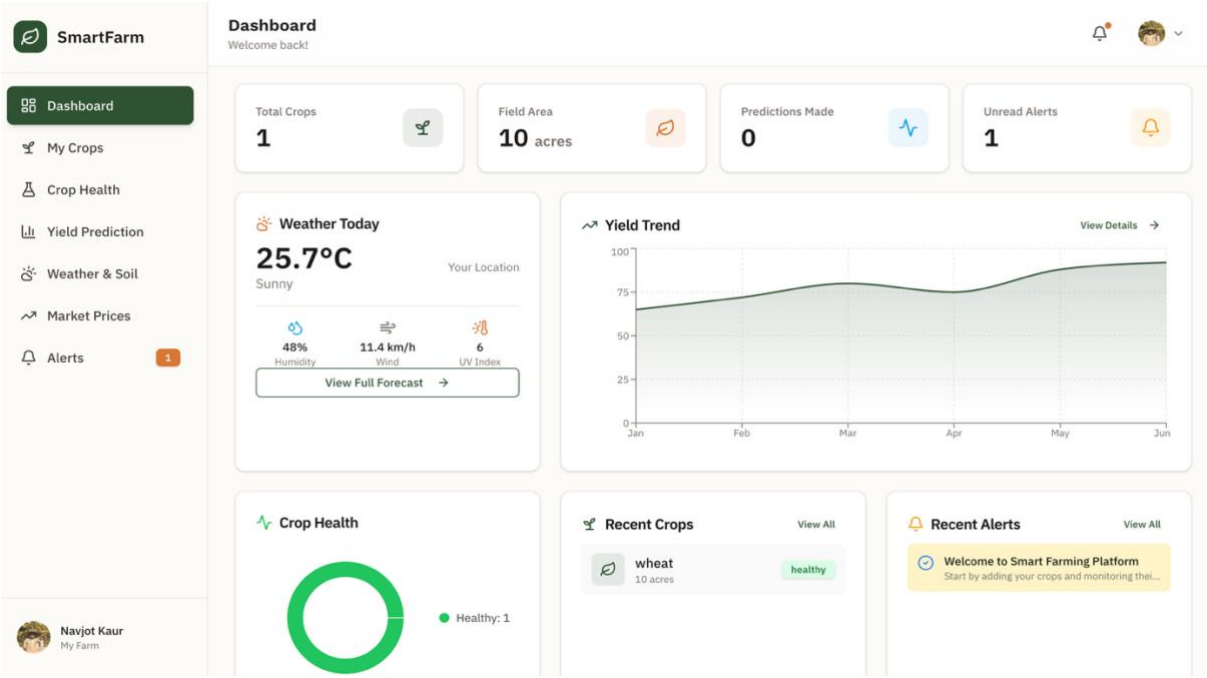

Figure 4: Dashboard Overview

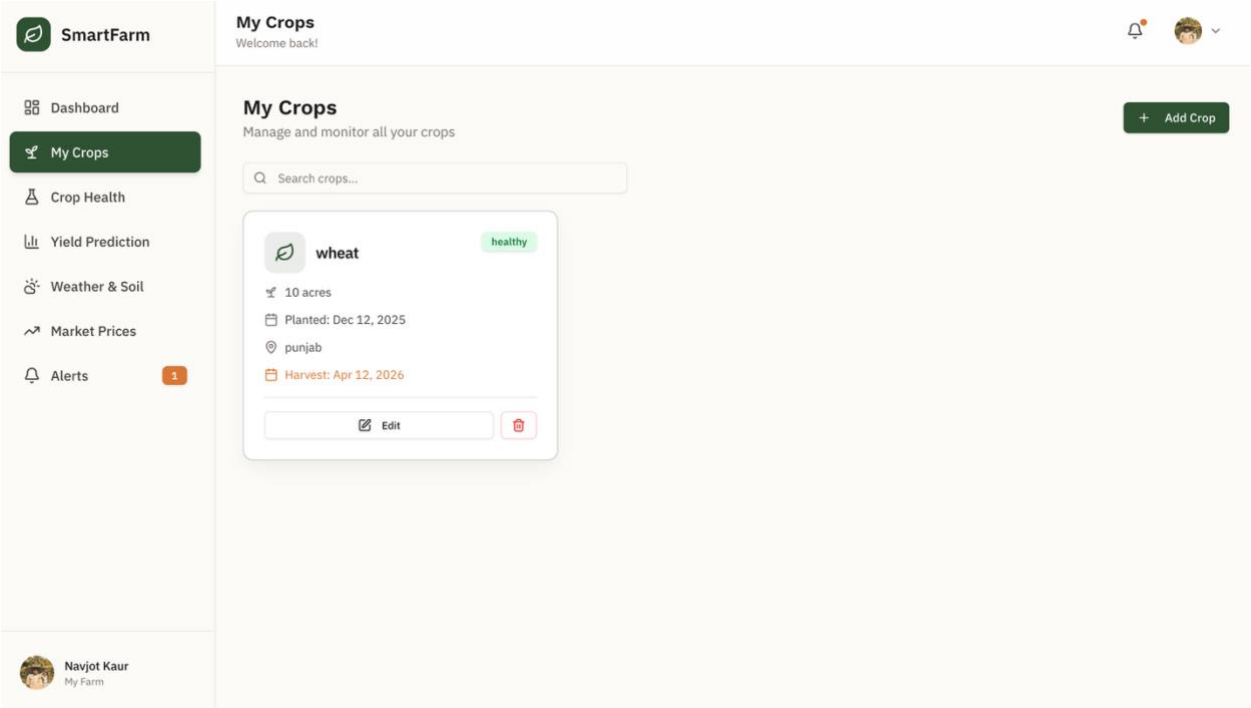

Figure 5: Crop Management Page

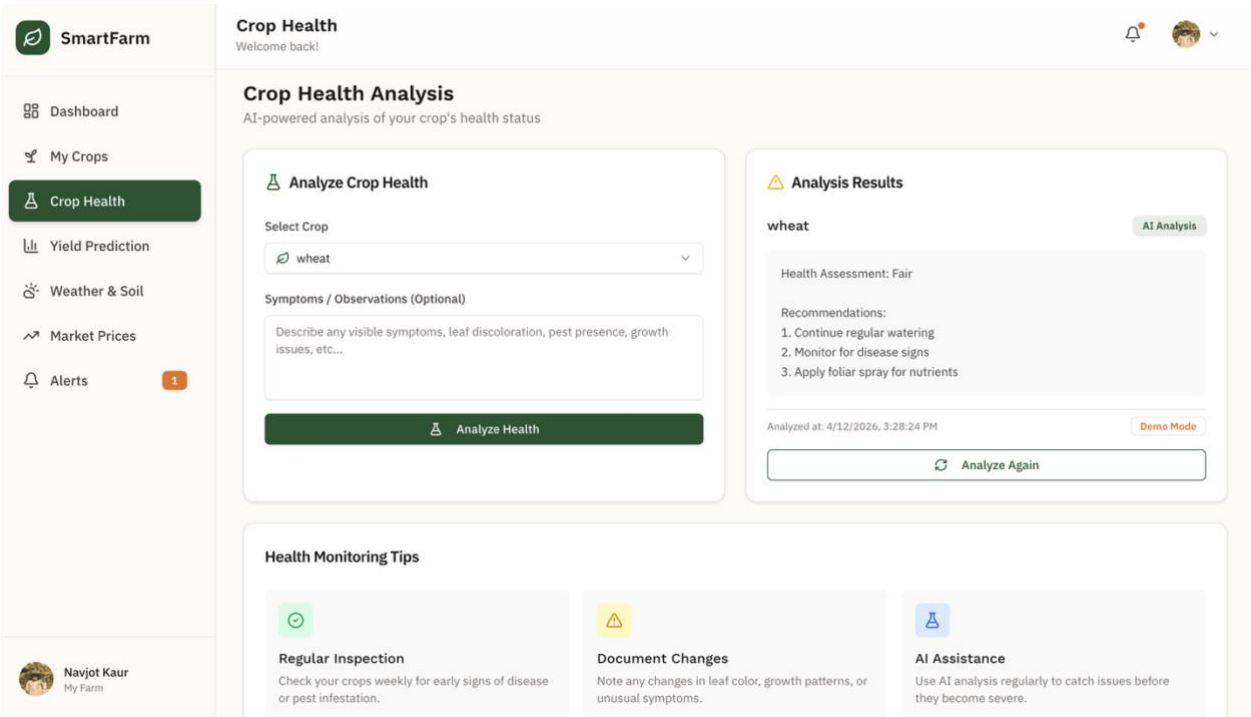

Figure 6: Crop health Analysis

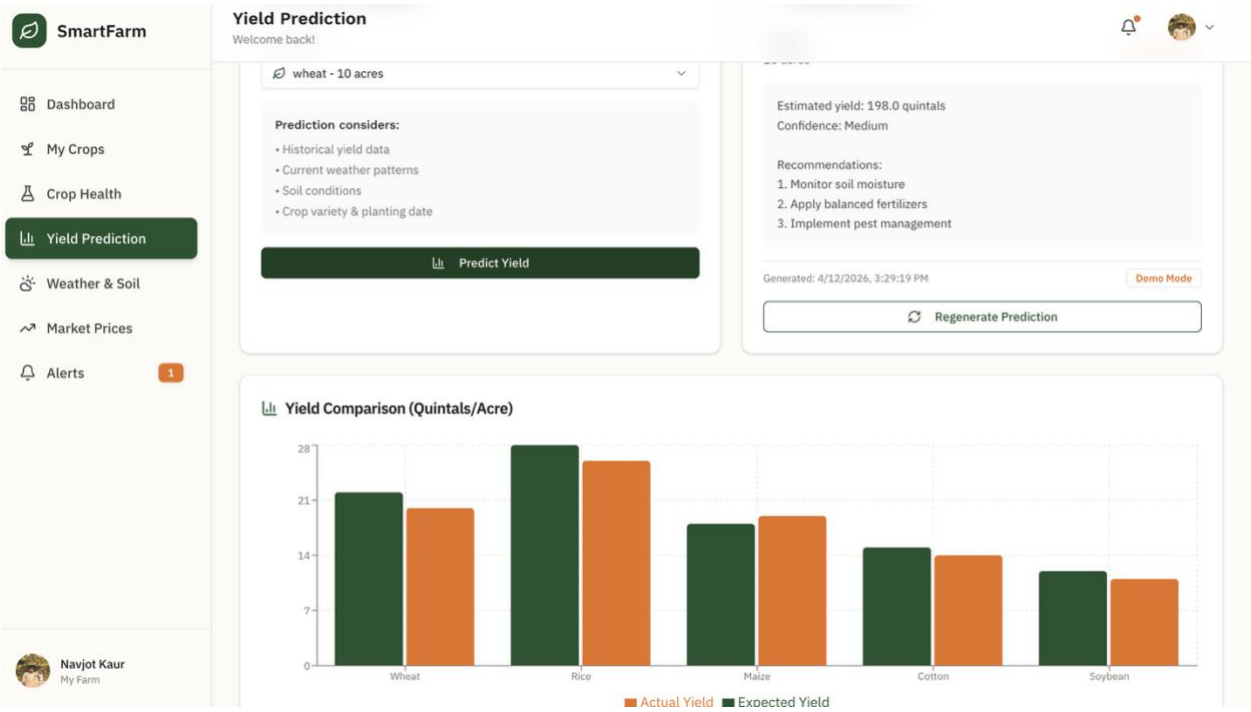

Figure 7: Yield Prediction Module

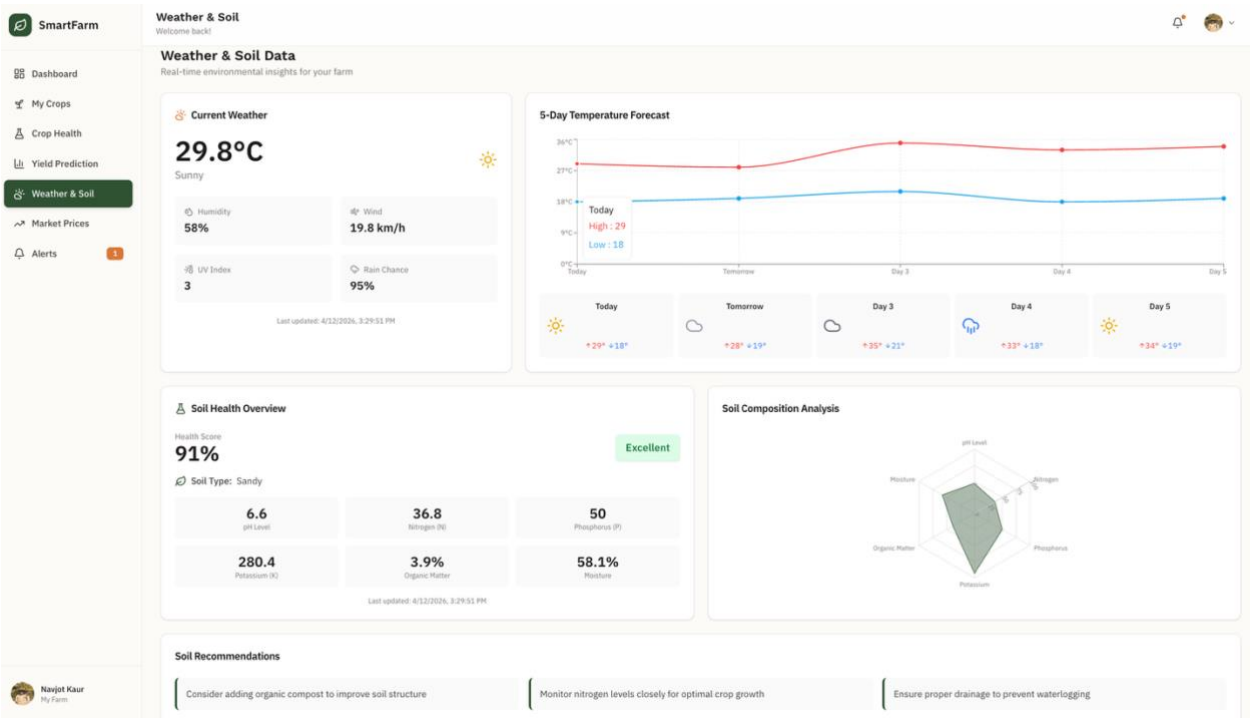

Figure 8: Weather and Soil Data

11. Glossary

- **AI:** Technology which allows computers to carry out tasks that usually need human thought like prediction and analysis.
- **API:** A way for separate computer programs and systems to exchange and transfer data with each other.
- **Crop Health Monitoring:** Watch crop health check any disease stress or growth issue
- **Dashboard:** A single dashboard with summarized view of essential information
- **FastAPI:** A python framework designed to build speedy and resource efficient backend APIs.
- **Frontend:** The application interface that the user will see.
- **Backend:** The application logic and datastores.
- **JWT (JSON Web Token):** A means of verifying and authenticating a user identity and granting the user access to certain areas of the system.
- **LLM (Large Language Model):** An advanced artificial intelligence (AI) that is trained on the skills of understanding opposing to human and natural language.
- **MVP (Minimum Viable Product):** A minimum viability of the product in the best of cases only basic features are presented in it.
- **React:** A JavaScript framework used to write front end software
- **PostgreSQL:** A very little file-based database inside the app.
- **Yield Prediction:** The process of forecasting crop production volumes.

GitHub and Deployment Link

Public Repository Link: https://github.com/NavjotKaur-CSP/SmartFarming.git

Live Application: https://smart-farming-kohl.vercel.app/

References

Patel, R., & Sharma, S. (2021). Smart farming using IoT and machine learning techniques. International Journal of Agricultural Technology, 17(2), 123–135.

Kumar, A., & Mishra, P. (2023). AI-based crop yield prediction using machine learning models. Journal of Smart Agriculture, 9(1), 45–58.

Zhang, C., & Kovacs, J. M. (2012). The application of small unmanned aerial systems for precision agriculture: A review. Precision Agriculture, 13(6), 693–712. https://doi.org/10.1007/s11119-012-9274-5

Mulla, D. J. (2013). Twenty-five years of remote sensing in precision agriculture: Key advances and remaining knowledge gaps. Biosystems Engineering, 114(4), 358–371. https://doi.org/10.1016/j.biosystemseng.2012.08.009

Jones, J. W., Antle, J. M., Basso, B., Boote, K. J., Conant, R. T., Foster, I., … & Wheeler, T. R. (2017). Toward a new generation of agricultural system data, models, and knowledge products. Agricultural Systems, 155, 269–288. https://doi.org/10.1016/j.agsy.2016.09.021